

Rapport d'optimisation

SAE 4.01 Zelda: Link's Awakening UPG

Dans le cadre du projet Zelda : *Link's Awakening UPG*, plusieurs optimisations ont été apportées à l'architecture technique, au front-end, au back-end et à l'environnement de déploiement. Ces ajustements ont permis de renforcer la performance, la qualité de code et la sécurité globale de l'application. Voici les différentes optimisations réalisées, avec un état des lieux *avant / après* et leurs impacts mesurables.

1) Optimisation du filtrage des markers

Dans la première version, chaque modification/ajout/suppression/filtrage d'un marqueur entraînait un rafraîchissement complet de l'application, provoquant des latences visibles et une pénibilité à utiliser l'application. Nous avons remplacé l'ancien système avec un nouveau basé sur JavaScript (côté client donc) qui, par exemple, active/désactive un marqueur. Ce changement a nettement amélioré la fluidité de la carte et l'expérience utilisateur. Là où nous avions une douzaine de rafraîchissement, nous en avons plus que deux : pour créer et mettre à jour un marqueur (compliqué à supprimer notamment car nous n'avons pas accès au popup individuel).

2) Réécriture du système de filtres combinés

Initialement, les filtres étaient organisés en formulaires classiques avec des relations mères/filles peu ergonomique et pénible à utilisés si l'utilisateur souhaitait combinés beaucoup de filtres (à cause des rafraîchissement). Nous avons conçu une logique de filtrage en JavaScript, basée sur des boutons agissant directement sur les marqueurs via une boucle efficace. Cette nouvelle approche permet un filtrage dynamique et combinatoire, bien plus performant et agréable pour l'utilisateur, surtout combiné au nouveau système d'ergonomie entre les relations mères/filles.

3) Suppression du système de refresh

Avec le nouveau système de rafraîchissement JavaScript, une part du code hérité contenait des fonctions de "refresh" ou de "reload" n'étaient plus nécessaires, ainsi que des fichiers obsolètes ou inutilisés. Ce code a été entièrement nettoyé, ce qui a permis d'alléger l'application et d'améliorer les performances globales, en plus de renforcer tout le système de l'application. Cela a également permis de simplifier la maintenance du projet.

4) Export automatique de la base de données

La sauvegarde de la base de données était jusque-là effectuée manuellement, ce qui rendait le processus incertain et risqué. Nous avons mis en place une tâche automatisée dans le conteneur Docker. Comment ça marche ? Eh bien quand un utilisateur se connecte à l'application, on vérifie si la différence de temps avec l'ancienne sauvegarde (contenu dans un fichier last_export.txt) est supérieur à 7 jours. Si c'est le cas, alors on va exporter au travers de l'utilisateur la base de données vers un fichier backup.sql. Cela renforce la fiabilité des sauvegardes et sécurise la conservation des données.

5) Dockerisation complète et sécurisation

Avant optimisation, l'application tournait sur un site peu sécurisé et sur lequel nous n'avions aucun contrôle, et avec des mots de passe faibles. Une dockerisation complète a été mise en place, accompagnée d'une sécurisation renforcée des formulaires (anti-CSRF), d'une validation des entrées, et d'une politique de mots de passe renforcée. Combiné à l'hébergement de la dockerisation sur le NAS sécurisé et rapide d'Enzo VANDEPOELE, on obtient un système 100% contrôlable et bien plus sécurisé.

6) Mise en place de tests fonctionnels et de performance

Le projet ne bénéficiait d'aucun test automatisé au départ. Cela nous prenait un temps considérable de vérifier un à un les fonctionnalités comme un utilisateur lambda. Pour pallier à ce problème, nous avons intégré des tests End-to-End avec Playwright, un audit de performance via Lighthouse, ainsi qu'un test de montée en charge avec Artillery. Cette batterie de tests permet d'assurer la robustesse du projet et de garantir un fonctionnement optimal, même en conditions de stress.

7) Hébergement public via DuckDNS et NAS personnel

L'application n'était pas accessible publiquement (de façon sécurisé). Pour simuler une montée en production, nous avons déployé le projet via un nom de domaine DuckDNS pointant vers un conteneur Docker sur NAS Windows 11. Nous avons ensuite ajouté des règles de redirections de ports et essayer, sans succès, d'ajouter une certification HTTPS avec Win ACME. Ce système permet un accès externe rapide et sécurisé et nous avons continué à essayer la mise en place d'une stratégie HTTPS.

Conclusion

L'ensemble de ces optimisations a permis de transformer un prototype fonctionnel mais fragile en une application largement plus fluide, robuste, maintenable aisément, et

accessible à tous de façon sécurisé et rapide. Chaque amélioration a été motivée par des besoins concrets identifiés pendant le développement ou les tests utilisateurs, et a abouti à des gains mesurables en performance, en fiabilité ou en expérience utilisateur.

Ce travail d'optimisation n'est donc pas seulement une amélioration à l'ancienne version, mais il constitue une refonte complète des fonctionnalités fragiles tout en s'appuyant sur des fonctionnalités et un visuel déjà très satisfaisant selon nous. En bref ce travail à constituait une étape clé dans la consolidation du projet Zelda : *Link's Awakening UPG*, et renforce l'ancienne version en **TOUS** point de vue.